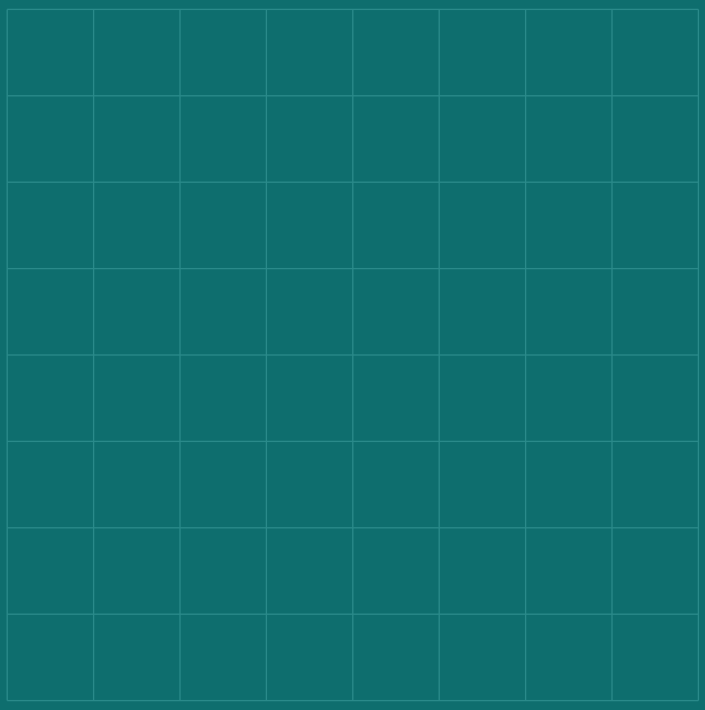


Geospatial Tile Server

Technical Documentation

On-the-fly Mapbox Vector Tiles from PostGIS, with caching, zoom-level simplification and per-layer access control.

Go · PostGIS · MVT · Redis

Version 1.0 · October 19, 2026

Contents

1. Introduction 5

1.1 Purpose . . . 5
1.2 Goals and non-goals . . . 5
1.3 Technology stack . . . 5
1.4 Terminology . . . 5

2. Architecture 7

2.1 Component overview . . . 7
2.2 Internal packages . . . 7
2.3 Design principles . . . 8

3. Request lifecycle 9

3.1 Step by step . . . 9
3.2 Latency budget (cache miss, z14 urban tile) . . . 9

4. Tile addressing 10

4.1 Coordinate system . . . 10
4.2 Bounds computation . . . 10
4.3 Resolution per zoom . . . 10

5. MVT generation 11

5.1 Query template . . . 11
5.2 Index usage . . . 11
5.3 Layer sources . . . 11
5.4 Attribute handling . . . 11
5.5 Limits and safeguards . . . 12

6. Zoom-level simplification 13

6.1 Why simplify . . . 13
6.2 Tolerance formula . . . 13
6.3 Strategies . . . 13
6.4 Zoom bands and precomputed generalization . . . 13
6.5 Feature filtering by zoom . . . 13

7. Caching 15

7.1 Layers of cache . . . 15
7.2 Cache key . . . 15
7.3 Layer versions . . . 15
7.4 Invalidation . . . 15

7.5 Stampede protection . . . 16
7.6 Pre-warming . . . 16
7.7 Redis sizing . . . 16
8. Access control 17
8.1 Model . . . 17
8.2 Authentication . . . 17
8.3 Policy file example . . . 17
8.4 Evaluation semantics . . . 17
8.5 Rate limiting . . . 18
8.6 Caching and authorization interplay . . . 18
9. Configuration 19
9.1 Sources and precedence . . . 19
9.2 Full example . . . 19
9.3 Key reference . . . 20
9.4 Hot reload . . . 21
10. HTTP API 22
10.1 Endpoints . . . 22
10.2 Tile response headers . . . 22
10.3 TileJSON example . . . 22
10.4 Invalidation API . . . 23
10.5 Errors . . . 23
10.6 Client integration (MapLibre GL JS) . . . 23
11. Data preparation 24
11.1 Table requirements . . . 24
11.2 Recommended setup . . . 24
11.3 Subdividing large geometries . . . 24
11.4 Database roles . . . 24
12. Deployment 25
12.1 Container image . . . 25
12.2 Local development (Docker Compose) . . . 25
12.3 Kubernetes . . . 25
12.4 Graceful shutdown . . . 25
12.5 Upgrades and migrations . . . 25
13. Performance tuning 27
13.1 Checklist . . . 27

13.2 Reference benchmarks . . . 27

14. Observability **28**

14.1 Metrics . . . 28

14.2 Suggested alerts . . . 28

14.3 Logging . . . 28

14.4 Tracing . . . 28

15. Testing **29**

15.1 Test layers . . . 29

15.2 Inspecting generated SQL . . . 29

16. Security considerations **30**

17. Troubleshooting **31**

Appendix A. CLI reference **32**

Appendix B. Repository layout **33**

Appendix C. Roadmap **34**

1. Introduction

1.1 Purpose

Geospatial-Tile-Server (internally **tilesrv**) is a vector tile service written in Go. It reads geometry directly from PostGIS tables and encodes it as Mapbox Vector Tiles (MVT) on request, so maps always reflect the current state of the database without a separate tile-baking pipeline. Generated tiles are cached in Redis, geometry is simplified per zoom level to keep tiles small, and every layer carries its own access policy.

This document is the technical reference for engineers who build, operate, extend or integrate with the service. It covers architecture, the tile generation pipeline, caching, simplification, access control, configuration, the HTTP API, deployment, observability, performance tuning and troubleshooting.

1.2 Goals and non-goals

Goals	Non-goals
Serve MVT tiles from live PostGIS data with p95 latency under 50 ms on cache hit and under 400 ms on miss	Raster tile rendering (PNG/JPEG)
Keep tile payloads under 500 KB at every zoom through simplification and feature limits	Editing or writing geometry back to PostGIS
Enforce per-layer, per-tenant authorization, including row-level filters	Acting as a general-purpose GIS query API
Scale horizontally with stateless instances	Storing styles or sprites (served by a CDN or style server)
Invalidate stale tiles precisely when data changes	Offline tile packaging (MBTiles/PMTiles export is a future item)

1.3 Technology stack

Component	Version	Role
Go	1.23+	Service runtime, HTTP server, orchestration
PostgreSQL	16	Relational store
PostGIS	3.4+	Geometry storage, ST_AsMVT encoding, ST_TileEnvelope, simplification
Redis	7.2+	Tile cache, version counters, rate-limit buckets
pgx/v5	5.x	PostgreSQL driver and connection pool
go-redis/v9	9.x	Redis client (standalone, sentinel, cluster)
chi/v5	5.x	HTTP router and middleware
Prometheus client	1.x	Metrics
OpenTelemetry	1.x	Distributed tracing

1.4 Terminology

Term	Meaning
Tile	A square region of the map at a given zoom, addressed as z/x/y in Web Mercator (EPSG:3857)
MVT	Mapbox Vector Tile: protobuf encoding of features in tile-local integer coordinates
Extent	The integer coordinate space of one tile, default 4096
Buffer	Extra pixels around the tile edge included to avoid clipping artifacts, default 64
Layer	A named source of features backed by a table, view or SQL function
Tileset	A named group of layers served together in one tile
Principal	The authenticated caller (user, service or anonymous)
Layer version	Monotonic counter bumped on data change; part of the cache key

2. Architecture

2.1 Component overview

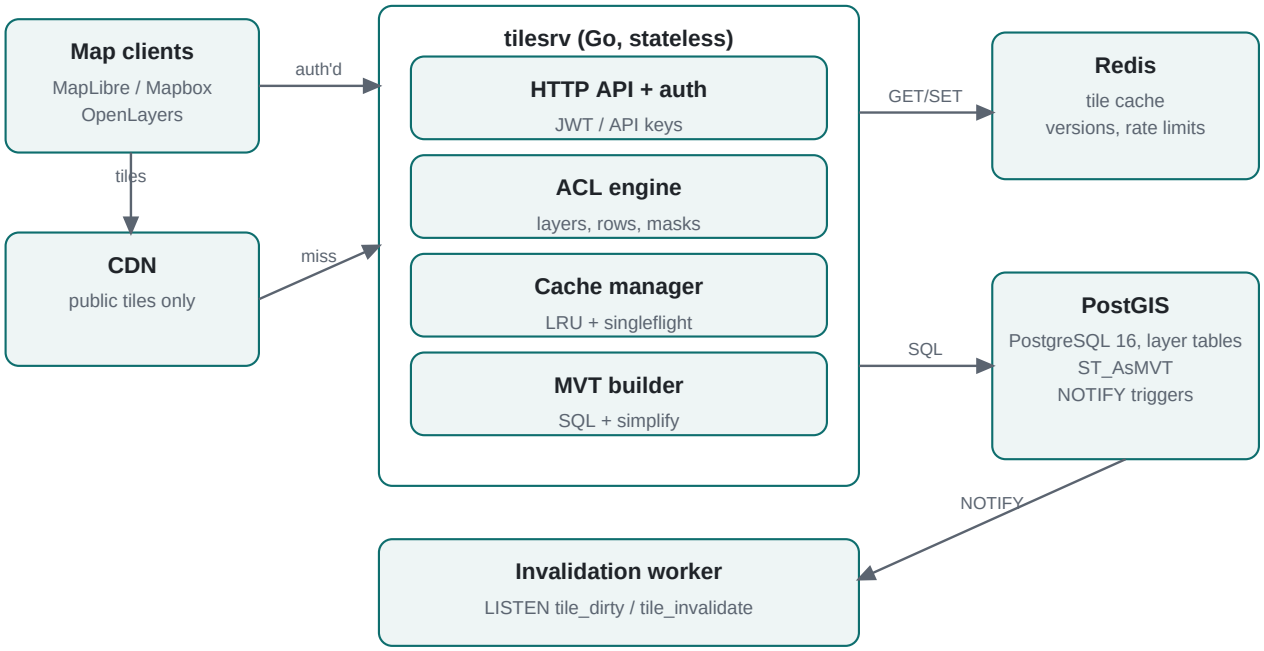


Figure 1. Component overview

The service is a single stateless Go binary. All shared state lives in PostgreSQL (data and layer metadata) and Redis (cache and counters). Any number of instances can run behind a load balancer; a CDN in front absorbs public, cacheable traffic.

2.2 Internal packages

Package	Responsibility
cmd/tilesrv	Entrypoint: flag parsing, config load, dependency wiring, graceful shutdown
internal/config	YAML + environment config loading, validation, hot reload of layer catalog
internal/httpapi	Routes, request parsing, response headers, error mapping
internal/auth	Token verification (JWT/OIDC, API keys), principal construction
internal/acl	Policy evaluation: layer visibility, zoom limits, row filters, attribute masks
internal/catalog	Layer and tileset registry, introspection of PostGIS columns and SRIDs
internal/tile	Tile math: bounds, validation, parent/child traversal
internal/mvt	SQL builder for ST_AsMVT queries, layer merge, gzip
internal/simplify	Per-zoom tolerance computation and simplification strategies
internal/cache	Redis cache, key construction, singleflight, negative caching
internal/invalidate	LISTEN/NOTIFY consumer, version bumps, dirty-tile purging

Package	Responsibility
internal/obs	Logging (slog), Prometheus metrics, OpenTelemetry spans

2.3 Design principles

- **Database does the geometry work.** Clipping, transformation, simplification and MVT encoding run inside PostGIS, close to the data and its indexes. Go orchestrates, authorizes and caches.
- **Authorization before cache.** The cache key includes the policy fingerprint, so a cached tile is never served to a caller with different permissions.
- **Versioned keys over deletes.** Invalidation bumps a layer version instead of scanning and deleting keys, which keeps invalidation O(1).
- **Fail closed.** If the policy engine cannot evaluate a request, the layer is omitted or the request is rejected; data is never served by default.
- **Bounded work per request.** Every query has a statement timeout, a feature limit and a byte budget.

3. Request lifecycle

3.1 Step by step

A request for GET /tiles/{tileset}/{z}/{x}/{y}.mvt passes through these stages:

1. **Routing and parsing.** The router extracts tileset, z, x, y and the format extension. Coordinates are validated: 0 <= z <= maxzoom and 0 <= x, y < 2^z. Invalid input returns 400.
2. **Authentication.** The auth middleware reads the Authorization header (Bearer JWT or API key) and builds a Principal. Missing credentials yield the anonymous principal.
3. **Policy evaluation.** For each layer in the tileset, the ACL engine decides: allow, deny, or allow-with-filter. It produces the effective layer list, row filters, attribute masks and a policy fingerprint (SHA-256 of the evaluated policy). If no layer is visible, the response is 403 (authenticated) or 401 (anonymous).
4. **Cache lookup.** The cache key is built from tileset, z/x/y, the versions of the effective layers and the policy fingerprint. A hit returns immediately.
5. **Coalescing.** On a miss, a singleflight group keyed by the cache key ensures only one goroutine per instance generates a given tile; concurrent requests wait for that result.
6. **Generation.** The MVT builder composes one SQL statement per tile that unions all layers, runs it with a statement timeout, and receives the encoded bytes.
7. **Post-processing.** Empty tiles are recorded with a short negative TTL. Non-empty tiles are gzipped (if not already) and written to Redis asynchronously.
8. **Response.** The handler sets Content-Type, Content-Encoding, ETag, Cache-Control and Vary headers and writes the body.

3.2 Latency budget (cache miss, z14 urban tile)

Stage	Typical	Upper bound
Parse + auth (JWT cached JWKS)	0.2 ms	2 ms
Policy evaluation	0.1 ms	1 ms
Redis GET (miss)	0.4 ms	5 ms
PostGIS query + encode	60-180 ms	statement_timeout (2 s)
gzip	2-6 ms	20 ms
Redis SET (async)	-	-
Total	**~90 ms**	**2 s**

4. Tile addressing

4.1 Coordinate system

Tiles follow the XYZ (Slippy Map) scheme in Web Mercator, EPSG:3857. The world spans -20037508.34 to 20037508.34 metres on both axes. At zoom z there are 2^z by 2^z tiles; y increases southward. TMS (y flipped) is supported via the ?scheme=tms parameter.

4.2 Bounds computation

The tile envelope is computed in Go for cache and logging, and in SQL with ST_TileEnvelope for the query:

```
const worldSize = 40075016.68557849
const origin    = 20037508.342789244

func Bounds(z, x, y int) (minX, minY, maxX, maxY float64) {
    size := worldSize / float64(uint64(1)<<z)
    minX = -origin + float64(x)*size
    maxX = minX + size
    maxY = origin - float64(y)*size
    minY = maxY - size
    return
}
```

4.3 Resolution per zoom

Ground resolution of one tile unit is worldSize / (2^z * extent). With extent 4096:

Zoom	Tile width (m)	Metres per tile unit
0	40,075,017	9,784
4	2,504,689	611.5
8	156,543	38.2
10	39,136	9.55
12	9,784	2.39
14	2,446	0.597
16	611.5	0.149
18	152.9	0.037

This table drives the simplification tolerances in section 6.

5. MVT generation

5.1 Query template

Each layer contributes a CTE. The template below shows one layer; the builder generates one block per effective layer and concatenates the results with the || operator, which is valid for MVT because a tile is a sequence of layer messages.

```
WITH
bounds AS (
  SELECT ST_TileEnvelope($1, $2, $3)              AS geom,
         ST_TileEnvelope($1, $2, $3, margin => $4) AS geom_buf
),
roads_src AS (
  SELECT t.id, t.name, t.class,
         ST_AsMVTGeom(
           ST_Simplify(ST_Transform(t.geom, 3857), $5, true),
           b.geom, 4096, 64, true) AS geom
  FROM public.roads t, bounds b
  WHERE t.geom && ST_Transform(b.geom_buf, 4326)
    AND t.min_zoom <= $1
    AND (t.tenant_id = $6)              -- row filter from ACL
  ORDER BY t.priority DESC
  LIMIT 20000
)
SELECT ST_AsMVT(roads_src.*, 'roads', 4096, 'geom', 'id')
FROM roads_src WHERE geom IS NOT NULL;
```

Parameters: $1..$3 are z/x/y, $4 is the buffer as a fraction of the tile (64/4096), $5 is the simplification tolerance in metres, $6 onwards are row-filter values. All user-influenced values are bound parameters; identifiers come only from the validated catalog and are quoted with pgx.Identifier.

5.2 Index usage

The && test against the buffered envelope uses the GiST index on the geometry column. To avoid a per-row ST_Transform on the column, the envelope is transformed to the table SRID instead. Tables stored in 3857 skip the transform entirely and are the fastest option; see section 13.

5.3 Layer sources

Source type	Configuration	Notes
table	schema, table, geometry column	Columns auto-introspected; attributes whitelisted in config
view	schema, view	Must expose an indexed geometry column from an underlying table
function	schema.function(z,x,y,params jsonb) returning bytea	Full control; function must return MVT bytes and respect bounds
sql	inline SELECT with {{bbox}} and {{tolerance}} placeholders	Reviewed at load time; placeholders replaced by bound parameters

5.4 Attribute handling

- Only attributes listed in the layer's properties block are emitted.

- Types map as: integer/bigint to int64, real/double to double, text/varchar to string, boolean to bool, jsonb is flattened one level or serialized as string.
- The feature id column (if configured) is written to the MVT feature id field so clients can use feature-state.
- Attribute masks from the ACL (section 8) remove columns before encoding, not after, so masked values never leave the database.

5.5 Limits and safeguards

Safeguard	Default	Config key
Max features per layer per tile	20,000	layers[].max_features
Statement timeout	2 s	postgres.statement_timeout
Max encoded tile size	1 MB (excess layers dropped by priority, warning logged)	tiles.max_bytes
Max zoom	22	tilesets[].max_zoom
Overzoom	Beyond source max_zoom, the parent tile is served for clients to overzoom	layers[].max_zoom

6. Zoom-level simplification

6.1 Why simplify

At low zoom a single tile covers a continent. Sending full-resolution coastlines or parcels produces multi-megabyte tiles and forces the GPU to draw vertices smaller than a pixel. Simplification removes vertices that would collapse to the same tile unit anyway.

6.2 Tolerance formula

The default tolerance is a multiple of the size of one tile unit at the requested zoom:

```
func Tolerance(z int, extent int, factor float64) float64 {
    unit := worldSize / (float64(uint64(1)<<z) * float64(extent))
    return unit * factor   // factor default 1.0, configurable per layer
}
```

A factor of 1.0 is visually lossless because sub-unit detail is snapped away by ST_AsMVTGeom anyway. Factors of 2-4 suit dense polygon layers at low zoom where slight smoothing is acceptable.

6.3 Strategies

Strategy	PostGIS function	Use for	Trade-off
douglas_peucker	ST_Simplify(geom, tol, true)	Lines and general use	Fast; can produce self-intersections in polygons
preserve_topology	ST_SimplifyPreserveTopology	Polygons that must stay valid	2-5x slower
visvalingam	ST_SimplifyVW(geom, tol*tol)	Natural features, coastlines	Smoother appearance; area-based tolerance
snap_grid	ST_SnapToGrid(geom, tol)	Points and very dense data	Cheapest; coarse
precomputed	Separate generalized tables per zoom band	Very large datasets (national parcels, OSM)	Extra storage; best runtime performance
none	-	Small layers or z >= max_zoom	Full fidelity

6.4 Zoom bands and precomputed generalization

For large tables, runtime simplification still has to read every vertex. The precomputed strategy routes each zoom band to a pre-generalized table:

```
layers:
  - name: parcels
    simplify:
      strategy: precomputed
      bands:
        - { zoom: [0, 8],   table: gen.parcels_z8,  }
        - { zoom: [9, 12],  table: gen.parcels_z12 }
        - { zoom: [13, 22], table: public.parcels  }
```

Generalized tables are built with a provided script (scripts/generalize.sql) and refreshed by the same change triggers that drive cache invalidation.

6.5 Feature filtering by zoom

Simplification is paired with feature thinning:

- **min_zoom column**: features carry a minimum zoom; the query filters min_zoom <= z.
- **area threshold**: polygons smaller than (k * unit)^2 are dropped, default k = 2.
- **priority ordering**: when max_features is hit, the highest-priority features are kept.
- **point clustering** (optional): at z below cluster_max_zoom, points are aggregated with ST_ClusterDBSCAN or a grid snap and emitted with a point_count attribute.

7. Caching

7.1 Layers of cache

Tier	Scope	TTL	Purpose
CDN / browser	Public tiles only	Cache-Control max-age (default 300 s), stale-while-revalidate 3600 s	Absorb global read traffic
In-process LRU	Per instance	30 s, 256 MB	Hot tiles, Redis offload during bursts
Redis	Shared	24 h default, per-layer override	Main tile cache
Negative cache	Shared	60 s	Empty tiles, avoids repeated empty queries over oceans

7.2 Cache key

```
tile:v1:{tileset}:{z}:{x}:{y}:{layerVersions}:{policyFP}

layerVersions = hex(xxhash64(join(sorted(layer + "@" + version))))
policyFP      = first 16 hex chars of SHA-256(evaluated policy)
```

Because versions and the policy fingerprint are part of the key, a data change or a different permission set naturally produces a different key. Old entries are never read again and expire through TTL. Callers with identical effective permissions share entries, so caching remains effective across many users in the same role or tenant.

7.3 Layer versions

Versions are stored in Redis as layerver:{layer} integers and mirrored in a local map refreshed every second and on invalidation events. A version bump is a single INCR.

7.4 Invalidation

Two mechanisms are supported and can be combined:

1. **Coarse (layer-wide).** An AFTER INSERT/UPDATE/DELETE statement trigger calls pg_notify('tile_invalidate', layer). The invalidate worker receives the notification and INCRs the layer version. Every tile of that layer becomes a miss. Simple and always correct; best for layers that change rarely.
2. **Fine (tile-level).** A row-level trigger writes the bounding box of the old and new geometry to tile_dirty(layer, bbox, created_at). The worker batches dirty boxes every 2 seconds, computes the covered tiles for each zoom in the cache range, and deletes those keys for the current version. Best for frequently edited layers where a full flush would cause a thundering herd.

```
CREATE OR REPLACE FUNCTION tilesrv.mark_dirty() RETURNS trigger AS $$
BEGIN
  INSERT INTO tilesrv.tile_dirty(layer, bbox)
  SELECT TG_ARGV[0], ST_Envelope(g)
  FROM (VALUES (CASE WHEN TG_OP <> 'INSERT' THEN OLD.geom END),
               (CASE WHEN TG_OP <> 'DELETE' THEN NEW.geom END)) v(g)
  WHERE g IS NOT NULL;
  PERFORM pg_notify('tile_dirty', TG_ARGV[0]);
  RETURN NULL;
END $$ LANGUAGE plpgsql;
```

Fine invalidation caps the number of keys deleted per batch (default 50,000); beyond that it falls back to a version bump.

7.5 Stampede protection

- singleflight per instance collapses concurrent misses for the same key.
- A Redis lock (SET NX PX 5000 on lock:{key}) optionally coordinates across instances for very hot tiles; waiters poll the cache for up to the lock TTL.
- Stale-while-revalidate: when a fine invalidation deletes a key, the previous body is kept under stale:{key} for 30 s and served while a background regeneration runs.

7.6 Pre-warming

tilesrv warm --tileset basemap --bbox -122.5,47.4,-122.2,47.8 --zooms 0-14 --concurrency 16 generates tiles for a region ahead of traffic. Pre-warming runs with the service principal's policy fingerprint; it only helps callers who share that fingerprint, typically public tilesets.

7.7 Redis sizing

Average tile size is around 40-80 KB gzipped for mixed basemap data. Budget memory as hot_tiles x avg_size x 1.2 overhead. Configure maxmemory-policy allkeys-lfu so the cache degrades gracefully.

8. Access control

8.1 Model

Authorization is evaluated per layer for every request. A policy binds subjects (roles, tenants, users or anonymous) to layers with optional constraints.

Constraint	Effect
allow / deny	Layer included or omitted. Deny wins over allow.
zoom range	Layer visible only within [min_zoom, max_zoom] for this subject
bbox / region	Layer clipped to a permitted geometry (e.g. a tenant's service area)
row filter	SQL predicate with bound claims, e.g. tenant_id = :claims.tenant
attribute mask	Columns removed from output for this subject
rate class	Selects a rate-limit bucket (section 8.5)

8.2 Authentication

- **JWT / OIDC**: RS256/ES256 tokens validated against a JWKS endpoint (cached, refreshed every 10 minutes or on unknown kid). Required claims: sub, exp, aud. Roles and tenant read from configurable claim paths.
- **API keys**: Opaque keys stored as SHA-256 hashes in tilesrv.api_keys with scopes and expiry. Passed as Authorization: ApiKey <key> or the ?key= query parameter for map clients that cannot set headers (query keys should be restricted by HTTP referrer).
- **Anonymous**: Allowed only for layers with an explicit public policy.

8.3 Policy file example

```
policies:
  - name: public-basemap
    subjects: [anonymous, "role:*"]
    layers: [water, landuse, roads, places]
    effect: allow

  - name: tenant-assets
    subjects: ["role:field_tech", "role:manager"]
    layers: [assets, work_orders]
    effect: allow
    row_filter: "tenant_id = :claims.tenant_id"
    min_zoom: 10

  - name: hide-cost-from-techs
    subjects: ["role:field_tech"]
    layers: [assets]
    mask_attributes: [replacement_cost, vendor_contract]

  - name: restricted-region
    subjects: ["role:contractor"]
    layers: [assets]
    region: "SELECT geom FROM tilesrv.regions WHERE id = :claims.region_id"
```

8.4 Evaluation semantics

1. Collect all policies whose subjects match the principal.
2. For each layer, any deny excludes it.
3. Multiple allows combine: zoom ranges union, row filters OR together, attribute masks intersect (a column is hidden only if every allowing policy hides it). Region constraints union.
4. The result is canonicalized (sorted, normalized) and hashed into the policy fingerprint.
5. Row filters and region SQL are parsed at load time against a restricted grammar (comparisons, AND/OR/NOT, IN, claim references); arbitrary SQL is rejected. Claim values are always bound parameters.

Optionally, row-level security in PostgreSQL can enforce the same filters in depth: tilesrv sets SET LOCAL tilesrv.tenant = $1 per transaction and RLS policies reference current_setting('tilesrv.tenant').

8.5 Rate limiting

Token buckets in Redis keyed by principal and rate class. Defaults: anonymous 50 req/s per IP, authenticated 200 req/s per subject, burst 2x. Exceeding the limit returns 429 with Retry-After.

8.6 Caching and authorization interplay

- Responses for non-public tiles carry Cache-Control: private, so CDNs do not store them.
- ETag includes the policy fingerprint.
- Vary: Authorization is set on all responses.

9. Configuration

9.1 Sources and precedence

Configuration is read from a YAML file (--config, default /etc/tilesrv/config.yaml). Any scalar can be overridden by an environment variable named TILESRV_ followed by the uppercase path with underscores, e.g. TILESRV_REDIS_ADDR. Flags override environment. Secrets should come from environment or files referenced with file:// URIs.

9.2 Full example

```
server:
  listen: ":8080"
  admin_listen: ":9090"      # metrics, pprof, health
  read_timeout: 5s
  write_timeout: 10s
  shutdown_grace: 20s
  cors_origins: ["https://maps.example.com"]

postgres:
  dsn: "file:///run/secrets/pg_dsn"
  max_conns: 40
  min_conns: 4
  statement_timeout: 2s
  application_name: tilesrv

redis:
  mode: standalone            # standalone | sentinel | cluster
  addr: "redis:6379"
  password: "file:///run/secrets/redis_pw"
  pool_size: 64
  default_ttl: 24h

tiles:
  extent: 4096
  buffer: 64
  max_bytes: 1048576
  gzip_level: 6
  negative_ttl: 60s
  local_lru_mb: 256

auth:
  jwks_url: "https://idp.example.com/.well-known/jwks.json"
  audience: "tilesrv"
  roles_claim: "realm_access.roles"
  tenant_claim: "tenant_id"
  api_keys: true

tilesets:
  - name: basemap
    layers: [water, landuse, roads, places]
    min_zoom: 0
    max_zoom: 16
  - name: operations
    layers: [assets, work_orders]
    min_zoom: 8
    max_zoom: 20

layers:
  - name: roads
    source: { type: table, schema: public, table: roads, geometry: geom, srid: 4326 }
    id_column: id
    properties: [name, class, oneway]
    min_zoom: 5
    max_zoom: 16
    max_features: 20000
    simplify: { strategy: douglas_peucker, factor: 1.0 }
    cache_ttl: 24h
    invalidation: fine
```

9.3 Key reference

Key	Default	Description
server.listen	:8080	Public HTTP address

Key	Default	Description
server.admin_listen	:9090	Admin endpoints; never expose publicly
postgres.max_conns	4 x CPU	Pool ceiling; keep under PgBouncer/PG limits
postgres.statement_timeout	2s	Per-tile query timeout
redis.default_ttl	24h	TTL for cached tiles
tiles.extent	4096	MVT extent
tiles.buffer	64	Buffer in tile units
tiles.max_bytes	1 MiB	Hard size cap per tile
tiles.local_lru_mb	256	In-process cache size; 0 disables
layers[].invalidation	coarse	coarse or fine
layers[].simplify.factor	1.0	Tolerance multiplier

9.4 Hot reload

SIGHUP or POST /admin/reload re-reads the layer catalog and policies. The new configuration is validated and introspected before swapping atomically; a failed reload keeps the running configuration and logs the error. Server, Postgres and Redis connection settings require a restart.

10. HTTP API

10.1 Endpoints

Method	Path	Description
GET	/tiles/{tileset}/{z}/{x}/{y}.mvt	Vector tile (also .pbf)
GET	/tiles/{tileset}/{z}/{x}/{y}.mvt?layers=a,b	Subset of layers
GET	/tiles/{tileset}.json	TileJSON 3.0 metadata, filtered to visible layers
GET	/layers/{layer}.json	Layer schema: attributes, types, zoom range, bounds
GET	/tilesets	Tilesets visible to the caller
GET	/healthz	Liveness (process up)
GET	/readyz	Readiness (Postgres and Redis reachable)
GET	/metrics	Prometheus (admin port)
POST	/admin/reload	Reload catalog and policies (admin port, admin token)
POST	/admin/invalidate	Invalidate a layer or bbox (admin port)
GET	/debug/pprof/*	Go profiling (admin port)

10.2 Tile response headers

Header	Value
Content-Type	application/vnd.mapbox-vector-tile
Content-Encoding	gzip (when client accepts it; otherwise decompressed)
ETag	"{xxhash of body}-{policyFP}"
Cache-Control	public, max-age=300, stale-while-revalidate=3600 or private, max-age=60
Vary	Authorization, Accept-Encoding
X-Tile-Cache	HIT-LOCAL, HIT, MISS, STALE
X-Tile-Layers	Comma-separated layers included

Empty tiles return 204 No Content, which map clients treat as an empty tile.

10.3 TileJSON example

```
{
  "tilejson": "3.0.0",
  "name": "basemap",
  "tiles": ["https://tiles.example.com/tiles/basemap/{z}/{x}/{y}.mvt"],
  "minzoom": 0,
  "maxzoom": 16,
  "bounds": [-180, -85.0511, 180, 85.0511],
  "vector_layers": [
    { "id": "roads", "minzoom": 5, "maxzoom": 16,
      "fields": { "name": "String", "class": "String", "oneway": "Boolean" } }
  ]
}
```

10.4 Invalidation API

```
POST /admin/invalidate
Authorization: Bearer <admin token>
Content-Type: application/json

{ "layer": "roads", "bbox": [-122.4, 47.5, -122.3, 47.7], "zooms": [10, 18] }
```

Omitting bbox bumps the layer version. Response: 202 with the number of keys queued for deletion.

10.5 Errors

Errors are JSON: {"error": {"code": "...", "message": "...", "request_id": "..."}}.

Status	Code	Cause
400	invalid_tile	z/x/y out of range or malformed
401	unauthenticated	Missing or invalid credentials for a non-public tileset
403	forbidden	No visible layer for this principal
404	not_found	Unknown tileset or layer
413	tile_too_large	Tile exceeds max_bytes even after dropping layers
429	rate_limited	Rate limit exceeded
503	unavailable	Postgres unavailable or pool exhausted
504	timeout	Statement timeout

10.6 Client integration (MapLibre GL JS)

```
map.addSource('ops', {
  type: 'vector',
  url: 'https://tiles.example.com/tiles/operations.json',
});
map.setTransformRequest((url) => ({
  url,
  headers: url.startsWith('https://tiles.example.com')
    ? { Authorization: `Bearer ${token}` } : {},
}));
map.addLayer({ id: 'assets', type: 'circle', source: 'ops',
  'source-layer': 'assets', paint: { 'circle-radius': 4 } });
```

11. Data preparation

11.1 Table requirements

- A geometry column with a declared type and SRID (geometry(MultiPolygon, 3857) preferred).
- A GiST index on the geometry column.
- A stable integer primary key if feature ids are needed.
- Valid geometries: run UPDATE t SET geom = ST_MakeValid(geom) WHERE NOT ST_IsValid(geom).

11.2 Recommended setup

```
ALTER TABLE public.roads
  ADD COLUMN geom_3857 geometry(MultiLineString, 3857)
  GENERATED ALWAYS AS (ST_Transform(geom, 3857)) STORED;

CREATE INDEX roads_geom_3857_gist ON public.roads USING gist (geom_3857);
CREATE INDEX roads_min_zoom ON public.roads (min_zoom);
CLUSTER public.roads USING roads_geom_3857_gist;
ANALYZE public.roads;
```

Storing geometry in 3857 removes per-tile transforms. Clustering by the spatial index improves page locality and can halve I/O for large tables.

11.3 Subdividing large geometries

Huge polygons (countries, oceans) are read in full for every tile they touch. Split them with ST_Subdivide(geom, 512) into a helper table to keep per-tile reads proportional to the tile area.

11.4 Database roles

```
CREATE ROLE tilesrv_reader LOGIN;
GRANT USAGE ON SCHEMA public, gen TO tilesrv_reader;
GRANT SELECT ON public.roads, public.assets TO tilesrv_reader;
ALTER ROLE tilesrv_reader SET statement_timeout = '3s';
```

The service needs only SELECT on layer tables, plus EXECUTE on function sources and LISTEN rights for invalidation. A separate migration role owns the tilesrv schema.

12. Deployment

12.1 Container image

```
FROM golang:1.23 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -trimpath -ldflags="-s -w" -o /tilesrv ./cmd/tilesrv

FROM gcr.io/distroless/static:nonroot
COPY --from=build /tilesrv /tilesrv
USER nonroot
EXPOSE 8080 9090
ENTRYPOINT ["/tilesrv", "serve"]
```

12.2 Local development (Docker Compose)

```
services:
  db:
    image: postgis/postgis:16-3.4
    environment: { POSTGRES_PASSWORD: dev }
    ports: ["5432:5432"]
  redis:
    image: redis:7.2
    command: ["redis-server", "--maxmemory", "512mb",
              "--maxmemory-policy", "allkeys-lfu"]
  tilesrv:
    build: .
    environment:
      TILESRV_POSTGRES_DSN: postgres://postgres:dev@db/postgres
      TILESRV_REDIS_ADDR: redis:6379
    volumes: ["./config.dev.yaml:/etc/tilesrv/config.yaml:ro"]
    ports: ["8080:8080", "9090:9090"]
    depends_on: [db, redis]
```

Run make dev-seed to load sample data (Natural Earth plus synthetic assets) and open http://localhost:8080/viewer for a debug map.

12.3 Kubernetes

- Deployment with 3+ replicas, readiness probe on /readyz, liveness on /healthz.
- Resources: start at 1 CPU / 512 MiB request, 2 CPU / 1.5 GiB limit (local LRU included).
- HorizontalPodAutoscaler on CPU (70%) or on tilesrv_http_inflight_requests.
- PodDisruptionBudget minAvailable: 2.
- Connect through PgBouncer in transaction mode; set postgres.max_conns so that replicas x max_conns stays below the pooler limit.
- Separate Service for the admin port with a NetworkPolicy restricting it to monitoring and operators.

12.4 Graceful shutdown

On SIGTERM the server stops accepting connections, fails readiness, waits for in-flight requests up to shutdown_grace, flushes pending cache writes and closes pools. Set the pod terminationGracePeriodSeconds above shutdown_grace.

12.5 Upgrades and migrations

Schema migrations for the tilesrv schema (api_keys, tile_dirty, regions) live in migrations/ and are applied with tilesrv migrate up before rollout. Cache key prefix tile:v1 is bumped when the encoding changes, which invalidates the whole cache cleanly.

13. Performance tuning

13.1 Checklist

1. Store geometry in 3857 or add a generated 3857 column.
2. Ensure GiST indexes exist and are used (EXPLAIN ANALYZE the generated query; see 15.2).
3. Subdivide large polygons.
4. Use precomputed generalization for layers over ~10M features.
5. Set min_zoom per feature to thin low-zoom tiles.
6. Whitelist only needed attributes; long text fields inflate tiles.
7. Raise the CDN max-age for public, rarely changing tilesets.
8. Pre-warm low zooms (0-8), which are few tiles but expensive.
9. Tune PostgreSQL: shared_buffers 25% RAM, effective_cache_size 70%, work_mem 32-64 MB, random_page_cost 1.1 on SSD, max_parallel_workers_per_gather 0-2.

13.2 Reference benchmarks

Measured on 8 vCPU / 32 GB, PostgreSQL 16 on the same network, OSM-derived Washington State extract (roads, buildings, landuse), mixed zoom 10-16 workload.

Scenario	Throughput	p50	p95	p99
Redis hit	18,000 req/s	1.1 ms	3.4 ms	7 ms
Local LRU hit	42,000 req/s	0.3 ms	0.9 ms	2 ms
Miss, 4326 storage	210 req/s	140 ms	410 ms	780 ms
Miss, 3857 storage + subdivide	520 req/s	62 ms	180 ms	330 ms
Miss, precomputed bands	760 req/s	41 ms	120 ms	240 ms

Treat these as relative guidance; repeat with your own data using make bench (k6 script in bench/).

14. Observability

14.1 Metrics

Metric	Type	Labels
tilesrv_http_requests_total	counter	tileset, status, cache
tilesrv_http_request_duration_seconds	histogram	tileset, cache
tilesrv_tile_bytes	histogram	tileset, zoom
tilesrv_tile_generation_seconds	histogram	tileset, zoom
tilesrv_layer_features	histogram	layer, zoom
tilesrv_cache_ops_total	counter	tier, op, result
tilesrv_singleflight_shared_total	counter	tileset
tilesrv_invalidations_total	counter	layer, mode
tilesrv_pg_pool_acquired / idle / waiting	gauge	-
tilesrv_acl_denied_total	counter	layer, reason
tilesrv_http_inflight_requests	gauge	-

14.2 Suggested alerts

- Cache hit ratio below 80% for 15 minutes on public tilesets.
- p95 generation time above 1 s for 10 minutes.
- Postgres pool waiting above 0 for 5 minutes.
- 5xx rate above 1%.
- Invalidation worker lag (oldest tile_dirty row) above 60 s.

14.3 Logging

Structured JSON via slog. Each request logs request_id, tileset, z/x/y, cache result, bytes, duration, layer count and principal subject (hashed). Tokens, API keys and row-filter values are never logged. Slow tiles (above 500 ms) log per-layer timings and feature counts.

14.4 Tracing

OpenTelemetry spans: http.request, acl.evaluate, cache.get, tile.generate (with child span per SQL query, statement text without parameter values), cache.set. Export via OTLP to the collector set in OTEL_EXPORTER_OTLP_ENDPOINT.

15. Testing

15.1 Test layers

Level	Tooling	Covers
Unit	go test	Tile math, tolerance, key building, policy evaluation and fingerprinting, SQL builder output
Integration	testcontainers-go (PostGIS, Redis)	End-to-end tile generation, invalidation triggers, RLS
Golden tiles	Decoded MVT compared to fixtures	Encoding regressions (geometry and attributes)
Property	rapid	Policy combination laws (deny wins, mask intersection), tile coverage of bboxes
Security	Custom suite	Cross-tenant leakage: every tenant fetches every tile and asserts no foreign ids
Load	k6	Throughput and latency targets in section 13

15.2 Inspecting generated SQL

tilesrv explain --tileset operations --tile 14/2625/5722 --as role:field_tech --claims tenant_id=42 prints the generated SQL and runs EXPLAIN (ANALYZE, BUFFERS) against the configured database.

16. Security considerations

- **SQL injection.** Identifiers come only from the validated catalog; all values are bound. Row filters use a restricted grammar.
- **Cross-tenant leakage.** Policy fingerprint in cache key, Cache-Control: private for non-public tiles, optional PostgreSQL RLS as defense in depth, and the leakage test suite in CI.
- **Denial of service.** Statement timeouts, feature limits, byte caps, rate limiting, max zoom enforcement and singleflight.
- **Secrets.** Loaded from files or environment, never logged; API keys stored hashed.
- **Admin surface.** Separate port, separate token, network-restricted.
- **Transport.** TLS terminated at the ingress; HSTS recommended. Internal traffic to Postgres and Redis should use TLS in shared networks.
- **Least privilege.** Read-only database role per section 11.4.

17. Troubleshooting

Symptom	Likely cause	Action
Tiles empty at all zooms	Wrong SRID in config, or geometry column null	Check /layers/{name}.json and SELECT ST_SRID(geom)
Clipped labels or seams at tile edges	Buffer too small	Increase tiles.buffer to 128 for wide line styles
Polygons with holes/spikes at low zoom	Douglas-Peucker produced invalid output	Switch layer to preserve_topology or visvalingam
Slow low-zoom tiles	Full scan of large geometries	Subdivide, precompute bands, set min_zoom
503 under load	Pool exhausted	Raise max_conns within pooler limits, add replicas, improve hit ratio
Stale data after edits	Triggers missing or worker lag	Verify triggers, check tilesrv_invalidations_total and tile_dirty backlog
Users see another tenant's features	Policy misconfigured	Run tilesrv explain --as, verify row_filter; enable RLS; rotate cache prefix
Low hit ratio	Too many distinct policy fingerprints	Move per-user filters to per-tenant/role filters where possible
413 tile_too_large	Dense data at zoom	Lower max_features, raise simplify factor, add min_zoom

Appendix A. CLI reference

Command	Description
tilesrv serve	Run the HTTP server
tilesrv migrate up/down	Apply tilesrv schema migrations
tilesrv validate --config f	Validate config and introspect layers without serving
tilesrv warm ...	Pre-generate tiles for a bbox and zoom range
tilesrv explain ...	Show SQL and query plan for one tile and principal
tilesrv invalidate --layer l [--bbox]	Trigger invalidation from the command line
tilesrv keys create --scopes ...	Create an API key (printed once)
tilesrv version	Print build information

Appendix B. Repository layout

Path	Contents
cmd/tilesrv/	Main package and CLI commands
internal/	Packages listed in section 2.2
migrations/	SQL migrations for the tilesrv schema
scripts/	generalize.sql, triggers.sql, seed data loaders
deploy/	Dockerfile, Compose, Helm chart
bench/	k6 load tests
testdata/	Golden tiles and fixtures
docs/	This documentation source

Appendix C. Roadmap

- PMTiles/MBTiles export for offline use.
- Per-feature cache invalidation via H3 index of dirty areas.
- Server-side style-aware layer pruning (only fetch layers a style uses at a zoom).
- GeoParquet and FlatGeobuf sources alongside PostGIS.
- Tile diffing for efficient client updates.